

110.11. Devant, derrière : le buisson sur Game Boy normale, devant puis derrière

Les deux buissons, sur la Game Boy d'origine : sans DEVANT, c'est le programme qui choisit, selon la place du héros — `sprite()` à gauche, `spriteDerriere()` à droite.

1. Ce qu'il faut comprendre

Les deux buissons du 110.6, mais sur la Game Boy d'origine, en quatre nuances de gris. Pas de couleur : ni `couleurFond()`, ni `teindre()`.

Ce qui est nouveau ici : la limite de la Game Boy normale. Elle n'a qu'une carte pour le décor : rien pour dire « cette case-ci passe devant ». DEVANT n'existe pas. Il ne reste qu'un réglage, sur le lutin : `spriteDerriere()` (ou DERRIERE).

Le problème : DERRIERE vaut pour tout le décor dessiné. Avec `spriteDerriere()` seul, le héros passerait derrière les deux buissons. Avec `sprite()` seul, devant les deux.

L'astuce : changer de côté selon la place. Le premier buisson est à gauche (pixels 32 à 47), le second à droite (pixels 96 à 111). Entre les deux, il n'y a que le ciel. Alors, à chaque image : si `x < 80`, le héros est posé avec `sprite()` (devant) ; sinon, avec `spriteDerriere()` (derrière). Il passe devant le premier buisson, puis derrière le second.

Pourquoi 80 : c'est au milieu du ciel, entre les deux buissons. Là, devant ou derrière, on voit le héros pareil (le ciel est vide, l'indice 0) : le changement ne se voit pas.

La différence avec la Color : ici, c'est ton programme qui décide, à chaque image, avec un `if`. Sur la Color (la leçon suivante), c'est la case qui le dit, une fois pour toutes, avec DEVANT : plus besoin de savoir où sont les buissons.

Les buissons n'ont aucun 0, comme au 110.6 : avec des coins vides, deux touffes côte à côte laisseraient un trou, et l'on verrait un morceau du héros y rester figé.

À toi : inverse : derrière le premier buisson, devant le second. Il suffit d'échanger les deux lignes du `if`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <spriteDerriere>` → [110.10. La fonction spriteDerriere\(\) — un lutin derrière le décor](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **671 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

```
démarrage          57 o ← la console, à l’allumage – la mise en route de la console
main()             187 o ← démarrage – ta fonction
VBlank              8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage       28 o ← image() – attend l’image suivante
AttendreVBlank      16 o ← démarrage – attend que l’écran soit libre
AttendreAcces       12 o ← EcrireTexte – attend de pouvoir écrire une case
AdresseCase         10 o ← poser() – trouve l’adresse d’une case de l’écran
EcrireTexte         10 o ← texte() – écrit un texte, lettre par lettre
RangerLutins        11 o ← démarrage – range les 40 lutins hors de l’écran
InstallerTransfert  15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles        85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage – vide les deux cartes de l’écran
Tuiles              192 o ← CopierTuiles – 10 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert    10 o ← InstallerTransfert – la petite routine qui copie les lutins
"GAME BOY NORMALE"  16 o ← texte() – un texte gravé
```

Lettres gravées : ABEGLMNORY

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```
sprite() × 2    47 o sur place, dans main()
poser() × 2     26 o sur place, dans main() + AdresseCase (10 o)
texte() × 1     11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1      6 o sur place, dans main() + AttendreImage (28 o)
images() × 1     2 o sur place, dans main()
```

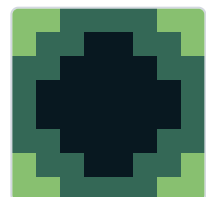
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME (Game Boy NORMALE, 4 nuances) : les deux buissons. Le héros
// avance tout seul : il passe DEVANT le premier, puis DERRIÈRE le second.
//
// Ce qui est nouveau : faire « devant / derrière » sans DEVANT.
// La Game Boy d'origine ne sait pas dire « cette case-ci passe devant ».
// Le seul réglage est sur le LUTIN, et il vaut pour TOUT le décor.
// Alors on change le lutin de côté SELON SA PLACE, à chaque image :
//
// x < 80 (à gauche, le buisson 1) -> sprite() : devant
// x >= 80 (à droite, le buisson 2) -> spriteDerriere() : derrière
//
// 80 est au milieu du ciel, entre les deux : le changement ne se voit pas.
//
// La leçon suivante fait la même chose sur Game Boy Color, avec poserDevant().
```

```
#include <Tuile>          // un dessin de 8 × 8 pixels
#include <poser>           // pose une tuile sur une case du fond
#include <texte>           // écrit un texte à l’écran
#include <sprite>          // place un lutin de 8 × 8 au pixel près (devant)
#include <spriteDerriere> // place un lutin derrière le décor
```

2. UN DESSIN — BUISSON

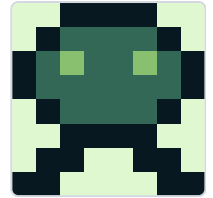
```
Tuile BUISSON = {          // Une touffe de feuilles : AUCUN 0, elle est toute pleine.
    "11222211",           // (des coins en 0 laisseraient un trou là où deux
    "12233221",           // touffes se touchent : on y verrait le héros)
    "22333322",
    "23333332",
    "23333332",
    "22333322",
    "12233221",
    "11222211",
};
```



à l’écran

3. UN DESSIN — HEROS

```
Tuile HEROS = { // 0 transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "3222223",
    "03222230",
    "00333300",
    "03300330",
    "3300033",
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    texte(1, 1, "GAME BOY NORMALE");
    for (uint8_t l = 8; l < 10; l++) { // Lignes 8 et 9 :
        for (uint8_t c = 4; c < 6; c++) { // buisson 1, colonnes 4 et 5,
            poser(c, l, BUISSON);
        }
        for (uint8_t c = 12; c < 14; c++) { // buisson 2, colonnes 12 et 13.
            poser(c, l, BUISSON); // Le même : c'est le héros qui changera de côté.
        }
    }

    uint8_t x = 0; // La place du héros, en pixels.

    while (true) { // La boucle du jeu :
        image();

        if (images() % 2 == 0) x++; // Un pixel toutes les 2 images.
        if (x > 152) x = 0;

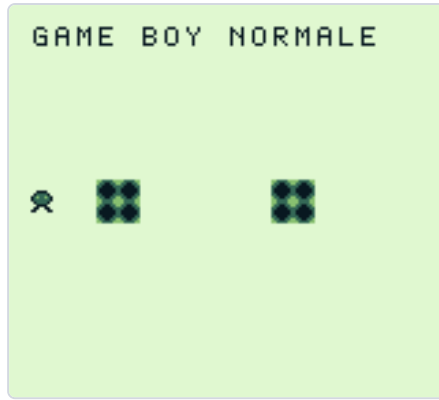
        // Devant ou derrière : on choisit selon la place du héros.
        if (x < 80) { // à gauche (le buisson 1) :
            sprite(0, x, 68, HEROS); // DEVANT le décor
        } else { // à droite (le buisson 2) :
            spriteDerriere(0, x, 68, HEROS); // DERRIÈRE le décor
        }
    }
}
```

3. Ce qui se passe, étape par étape

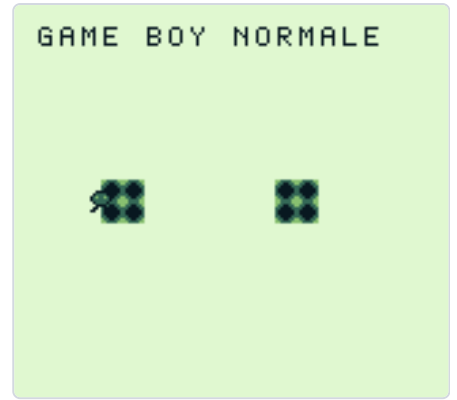
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



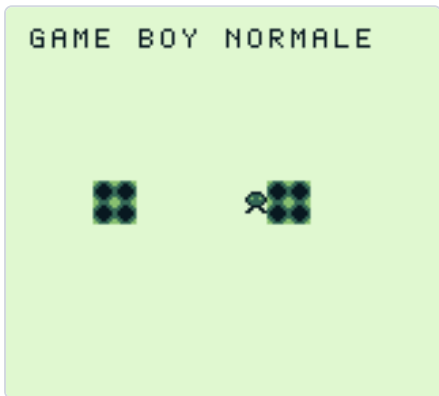
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Deux buissons gris. Le héros passe devant celui de gauche, puis disparaît derrière celui de droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche est en 4 nuances, sans couleur
- ✓ buisson 1 : le héros est devant (nuance 3)
- ✓ entre les deux, sur le ciel, on le voit (nuance 3)
- ✓ buisson 2 : le héros est derrière (nuance 1)

Ensuite : **110.12. La fonction `poserDevant()` — une case qui passe devant les lutins** — chapitre 17 : Le mouvement : les lutins au pixel près. · [gameboy3](#) — écrire une cartouche Game Boy en C++